

Programme de colle - Semaine 5 (du 21 septembre)

La colle est constituée d'une démonstration ou exercice de cours (parmi les énoncés ci-dessous marqués par une ()) (20min max) suivi d'un ou plusieurs exercices portant sur le programme ci-dessous.*

1 Langages formels. Langages réguliers

NB : les automates n'ont pas encore été abordés, on se concentre sur la définition inductive des langages réguliers pour commencer et la manipulation des mots et langages

- Mots : alphabet, mot sur un alphabet, concaténation, puissance d'un mot, structure de monoïde $(\Sigma^*, .)$.
- Préfixes, suffixes, facteurs, sous-mots.
- Langages : opérations ensemblistes sur les langages (union, intersection, différence, complémentaire), concaténation de deux langages, étoile de Kleene d'un langage,
- Langages réguliers (aussi appelés langages rationnels): définition inductive de la classe des langages réguliers, **tout langage fini est régulier (*)**
- Expressions régulières : définition inductive, langage dénoté par une expression régulière (notation $\mu(e)$)
- **Un langage est régulier si et seulement si il existe une expression régulière qui le dénote (*)** [NB : interroger sur un seul sens uniquement, chaque sens se démontre par induction]
- Expressions régulières équivalentes
- Expressions régulières étendues : par exemple $e?$, $e+$, $e\{3, 5\}$, etc. Aucune syntaxe n'est exigible mais il faut savoir comprendre une expression régulière étendue et la traduire en expression régulière du cours équivalente (ex : $e? \equiv (e|\epsilon)$).

2 Parcours de graphes

Les élèves doivent pouvoir lire, écrire ou modifier un algorithme de parcours de graphe en pseudo-code, en C ou en OCaml

- Définitions usuelles : graphe orienté, non orienté, sommet/nœud, arête/arc, voisins (entrants/sortants), degré, chemins, cycles, graphe pondéré, etc.
- Représentation par matrice d'adjacence ou listes d'adjacence.

- Distance entre deux sommets x et y , définie comme le poids d'un chemin de poids minimal, notée $\delta(x, y)$
- Parcours de graphes (voir le document : "tous les algos" sur la page du cours)
 - Parcours générique (Algo 1)
 - Parcours en profondeur : avec une pile (Algo 2) ou par récursivité (Algo 3)
 - Parcours en largeur : avec une file (Algo 4) et en version simplifiée (Algo 5)
 - Parcours précédents avec construction de l'arborescence du parcours (Algos 6, 7, 8)
- **Algorithme de Dijkstra**
 - Algorithme fonctionnant sur des graphes pondérés avec des **poids positifs** implémenté à l'aide d'une file de priorité (Algo 9)
 - **L'algorithme de Dijkstra garantit l'invariant suivant (*) :**

$$(I) \quad \forall x \in S, \quad x \text{ est fermé} \Rightarrow g[x] = \delta(x_d, x)$$

avec $g[x]$ la valeur affectée à x par l'algorithme de Dijkstra et x_d le sommet de départ choisi. En particulier cela justifie que l'algorithme de Dijkstra construit une arborescence de chemins optimaux (pour les sommets fermés).

- **Algorithme A* : recherche informée**
 - L'algorithme A* a pour objectif de trouver un chemin optimal entre un sommet de départ x_d et un sommet cible x_t .
 - L'algorithme A* utilise une *fonction heuristique* $h : S \rightarrow \mathbb{R}^+$ telle que $h(x)$ est une estimation de la distance à de x à x_t . Exemples vus en cours : distance euclidienne (à vol d'oiseau) lorsque le graphe est inclus dans le plan euclidien (ex : carte routière) et distance de Manhattan (villes quadrillées, grilles, etc). L'heuristique vérifie $h(x_t) = 0$.
 - Algorithme et illustration présentés en cours (Algo 10) : implémenté à l'aide d'une file de priorité : je demande aux élèves pour l'instant de savoir exécuter l'algorithme à la main sur un petit graphe et de bien comprendre les différences avec l'algorithme de Dijkstra (choix du sommet ouvert qui minimise $f = g + h$, réouverture possible des sommets fermés, arrêt lorsqu'on ferme x_t).
 - **Pas encore vu (ne pas interroger dessus) :** heuristique admissible, heuristique monotone, preuve de A*, conséquences quand l'heuristique est monotone
 - Heuristiques admissible et monotone

- ★ Définitions
- ★ **Si une heuristique est monotone alors elle est admissible (*)**
- ★ Si l'heuristique est admissible l'algorithme A* construit un chemin optimal.
- ★ Si l'heuristique est de plus monotone, il n'est pas nécessaire de réouvrir les sommets fermés.

3 Automates finis

Il s'agit pour l'instant de se familiariser avec cette nouvelle notion, par exemple en demandant un automate pour reconnaître un langage ou inversement en étudiant le langage des mots acceptés par un automate donné.

- Automate fini déterministe (afd) : définition, notion de calcul dans un afd, notion de blocage.
- Langage reconnu (ou accepté) par automate : calcul réussi, mots acceptés. Classe des langages reconnaissables.
- Accessibilité et co-accessibilité; afd émondé
- Afd complet; complétion d'un afd
- **Automate complémentaire : Si L est reconnaissable alors $\bar{L}$ l'est (*)**

NB : le lien entre langages réguliers et langages reconnaissables n'a pas encore été étudié NB : l'automate produit et les propriétés de clôture qui en découlent n'ont pas encore été étudiés